```cpp

// COS30008 - Midterm 2026

#pragma once

#include <cctype>

template<size_t N>
class KeyIterator
{
public:

    using difference_type = std::ptrdiff_t;
    using value_type = char;
    using iterator = KeyIterator;

    KeyIterator(const char aInit[N]) noexcept
    {
        for (size_t i = 0; i < N + 1; i++)
        {
            fKeys[i] = std::isalpha(aInit[N] - 'A');
        }
        fIndex = 0;
        fUpdateIndex = N;
    }

    size_t operator*() const noexcept
    {
        return fKeys[fIndex];
    }

    iterator& operator++() noexcept
    {

        fIndex++;
        if (fIndex > N + 1) {
            fIndex = 0;
        }

        return *this;

    }

    iterator operator++(int) noexcept
    {
        iterator lChange = *this;
        (*this)++;
        return lChange;

```

```
    }

    iterator& operator+=(char aKey) noexcept
    {
        fKeys[fUpdateIndex] = std::toupper(aKey - 'A');
        fUpdateIndex++;
        if (fUpdateIndex >= N + 1) {
            fUpdateIndex = 0;
        }

        return *this;
    }

private:

    char fKeys[N+1];
    size_t fIndex;
    size_t fUpdateIndex;
};

```